

시험_예제 정리

1. 리스트

Problem 01

연결 리스트 구현 시 더미 헤드를 사용하는 장단점을 말해보시오.

- 코드가 간결해지고, 일반화 가능하다.
- 메모리를 추가로 먹는다
- 연산 추가 작업이 소모된다.
- 직관성이 떨어진다.

Problem 02

때로는 원소 x 가 존재하는지 체크해주는 메서드 `contains()`도 요구한다. 이것은 본문에서 소개한 메서드 `index(x)`를 호출해서 간단히 해결할 수 있다. 다음 부분을 완성하시오.

```
def contains(self, x)-> bool :  
    if self.index(x)>=0 :  
        return True  
    else :  
        return False
```

Problem 03

연속된 원소들

3절의 연결 리스트에서 $i \sim j$ 번 원소(첫 원소는 0번 원소라 부른다)를 프린트하는 메서드 `printInterval()`을 작성하시오. [코드 5-15]의 클래스 `LinkedListBasic`에 메서드 `printInterval()`을 더하는 방식으로 하고 다음 부분을 완성하시오.

```
def printInterVal(self, i:int, j:int ) :  
    curr= self.__head.next # 더미노드 있는 경우  
    for a in range(j+1):  
  
        if (a>= i and a<=j) :
```

```
print(f"{a}번째 원소 : {curr.data}")
curr=curr.next
```

Problem 04

03번 문제와 똑같은 작업을 [코드 5-22]의 CircularLinkedList에서 하시오. 역시 클래스 CircularLinkedList에 메서드를 더하는 방식으로 하고 다음 부분을 완성하시오.



```
def printInterval (self, i : int , j:int ):
    curr= self.__tail.next # 더미노드 없는 버전
    for a in range(j+1):

        if (a>= i and a<=j) :
            print(f"{a}번째 원소 : {curr.data}")
            curr=curr.next
```

Problem 05

본문과 달리, 리스트의 원소 수를 확인하기 위해 변수 __numItems를 두지 않고 대신 비현실적이지만 메서드 numItems()로 구현했다고 가정한다. 다음 부분에 메서드 numItems()를 구현하되 두 가지 방법으로 구현하시오.

1. 재귀를 사용하지 않는 알고리즘으로 구현하시오.
for문. if (next == None): return.
2. 재귀 알고리즘으로 구현하시오.

```
# 비재귀 더미노드 없는 버전.
def numItems(self) :
    num=0
    curr= self.__head #더미노드 없는 버전
    while(curr is not None) :

        num +=1
        curr=curr.next
    return num

#재귀
def numItems(self, headNode) :
    if (headNode==None){
        return 0
    }
```

```
else :  
    return 1+self.numItems(headNode.next)
```

Problem 06

메서드 `pop()`에 인자를 하나 더 주어 `pop(index, k)`와 같이 호출하고, `index`번부터 연속된 `k`개의 원소 (`index`번 원소를 포함하여 `k`개)를 삭제하고자 한다. `pop(index, 1)`을 부르면 원래의 `pop(index)`와 같은 일을 하게 될 것이다. 만일 `index`번 노드부터 시작해서 남은 노드가 `k`개가 안 되면 지울 수 있는 최대 한도인 마지막 노드까지만 지운다. 다음 부분을 채워 넣어 이에 맞게 변형해보시오.

```
# 연결리스트 상정  
def pop(self, index : int , k : int ):  
    prev=self.__getNode(index-1)  
    # prev가 None이거나 prev.next가 없으면 삭제할 노드가 없는 것  
    if prev is None or prev.next is None: return  
  
    curr= prev.next #삭제할 것.  
  
    for i in range(k) : #k개 삭제해야하니까.  
        if(curr ==None) :  
            break  
  
        # 삭제 로직: prev의 다음을 curr의 다음으로 연결  
        # (curr를 건너뛴)  
        prev.next = curr.next  
        self.__numItems -= 1  
  
        # 다음 삭제할 노드로 이동  
        curr = prev.next  
  
    #종료조건  
    if prev.next is None :  
        break
```

Problem 07

다음은 5절의 양방향 연결 리스트를 파이썬으로 구현한 코드다. 이 리스트에서는 int 타입의 원소들이 오름차순으로 정렬된 형태로 유지된다. 이 양방향 연결 리스트에 임의의 정수 x를 삽입하는 메서드인 add()를 작성해보시오. 다음 부분을 채워 넣으면 된다.

```
class CircularDoublyLinkedList :
    def __init__(self):
        self.__head = BidirectNode("dummy", None, None)
        self.__head.prev= self.__head
        self.__head.next= self.__head
        self.__numItems = 0

    def add(self, x) -> None:
        newNode = BidirectNode(x, None, None)
        curr = self.__head.next # 첫 번째 노드부터 시작

        # 1. 삽입할 위치 찾기: x보다 크거나 같은 데이터를 가진 노드를 찾음
        # (더미 노드로 다시 돌아오기 전까지 반복)
        while curr != self.__head and curr.data < x:
            curr = curr.next

        # 2. 연결 고리 만들기 (포인터 4개 조정)
        # 이제 curr는 x가 들어갈 위치 바로 '뒤'의 노드가 됩니다.
        newNode.next = curr
        newNode.prev = curr.prev

        # 중요: 주변 노드들이 newNode를 가리키게 함
        curr.prev.next = newNode
        curr.prev = newNode

        # 3. 개수 증가
        self.__numItems += 1
```

Problem 08

두 노드의 레퍼런스 node1과 node2가 있다. 이들은 [코드 5-15]의 클래스 LinkedListBasic의 노드들인데 두 노드가 같은 연결 리스트에 속하는지는 모른다. 이들이 같은 연결 리스트에 속하는지 확인하는 코드를 작성하시오.

```
# 아이디어 :  
# 1. node1부터 끝까지 탐색하면서 node2 찾아  
# 2. node1부터 끝까지 탐색하면서 node1 찾아
```

```
def isInSameList(self) -> bool :  
    if (node1 == node2) : return True  
  
    curr= self.__node1  
    while(curr is not None):  
        if(curr == node2) : return True  
        curr=curr.next  
    curr=self.__node2  
    while(curr is not None):  
        if(curr == node1) : return True  
        curr=curr.next  
    return False
```

리스트는 앞에서 체크

Problem 09

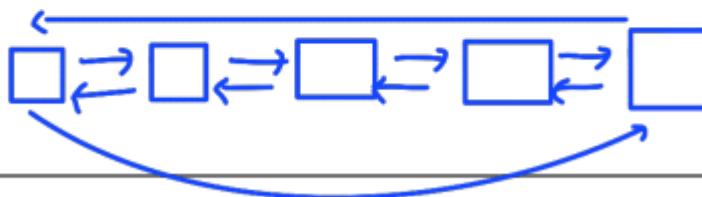
리스트에서 원소 x를 찾되 맨 마지막에 있는 x의 위치를 요구하는 메서드 `lastIndexOf(x)`가 필요하다. [코드 5-15]의 클래스 `LinkedListBasic`에 다음 메서드를 추가하시오. 다음 부분을 완성하면 된다.

```
# 더미 노드 없는 버전.  
def lastIndexOf(self, x):  
    # 아이디어 : 노드를 전부 순회하며 계속 x의 위치를 갱신.  
    lastIndex=-1  
    curr=self.__head  
    for i in range(self.__numItems) :  
        if(curr.data == x ) : lastIndex= i  
    return lastIndex
```

Problem 10

Circular에서는 뒤에서 체크

09번의 메서드 `lastIndexOf(self, x)`를 [코드 5-25]의 클래스 `CircularDoublyLinkedList`에 추가하시오.



```
def lastIndexOf(self, x) :
```

```
def lastIndexOf(self, x): 더미 x
    #아이디어 : 노드를 전부 순회하며 계속 x의 위치를 갱신
    #단, 다시 head로 돌아오면 종료
    lastIndex = -1
    curr = self.__head
    for i in range(self.__numItems)
        if(x == curr.data) :
            lastIndex = i
        curr= curr.next
```

뒤에서 부터 찾는 방식도 있다.

```
def lastIndexOf(self, x): 더미 x
    lastIndex = -1
    curr = self.__head.prev
    for i in range(self.__numItems) :
        if(x == curr.data) :
            lastIndex = i
    curr= curr.prev
```

시험에 안낼수준이다.

Problem 11



클래스 **LinkedListBasic**에 대하여, 리스트를 뒤집는 재귀 메서드를 구현하시오.

Q.이걸 앞에서 시작하는게 종료조건 좋지않나?

```
# 시험에 안낼 문제 수준이다.
def reverse(self) : # 더미 노드 x
    prev = self.__head
    if(prev.next is None ) : return

    for i in range(self.__numItems-1,0,-1) : # 3개의 노드이면 2번진행.
        prev = self.__getNode(i) #none가져갈때
        curr = prev.next #바꿀때

        curr.next=prev
        prev.next=None
```

2. 스택 예제 1

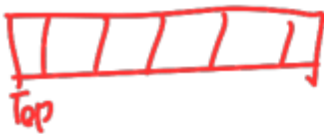
Problem 01

스택을 이용하여 다음과 같은 후위 표현법의 식을 계산하는 과정에서 스택의 상태 변화를 설명해보시오. (단, 정수는 한번에 읽어들이는다고 가정한다.)

15 25 + 10 2 * -

```
stack = [15,25]
operator +
stack = [40]
stack = [40,10]
stack = [40,10,2]
operator *
stack = [40,20]
operator -
stack = [20]
```

바뀌었습니다



Problem 02

본문에서 파이썬 내장 리스트를 사용하는 스택을 구현할 때 리스트의 맨 뒤를 스택의 탑으로 간주 했다. 반대로 리스트의 맨 앞을 스택의 탑으로 간주하여 다음 클래스 ListStack 코드를 바꾸어보시오.

```
class ListStack:
    def __init__(self):
        self.__stack = []

    def push(self, x):
        self.__stack.append(x)

    def pop(self):
        return self.__stack.pop()

    def top(self):
        if self.isEmpty():
            print("No element in stack")
        else:
            return self.__stack[0]

    def isEmpty(self) -> bool:
        return not bool(self.__stack)

    def popAll(self):
        self.__stack.clear()

    def printStack(self):
        print("Stack:")
        for i in range(len(self.__stack)):
            print('stack[', i, ']:', self.__stack[i])
```

insert(0, x)

pop(0)

0

→ range(self.__stack, -1, -1)

Problem 03

입력으로 들어온 문자열이 다음 집합의 원소인지 체크하는 코드를 스택을 이용하여 작성하시오.

$\{w\$w^R \mid w: \text{문자열}, w^R: w \text{의 순서를 뒤집어 놓은 것}\}$

1. \$ 전까지 w를 스택에 넣는다.

아이디어 \$직전까지 w를 스택에 넣는다.

스택에서 문자를 하나씩 빼면서 남은 문자열

```
def checker ( str) :  
    stk = __stack[]  
    dollarIndex = -1;  
    for i in range(len(str)):  
        ch=str[i]  
        if(ch is '$') :  
            dollarIndex=i  
            break;  
        else :  
            stk.push(ch)  
    for i in range(dollarIndex+1, len(str),+1) :  
        if(stk.isEmpty()): return False # 스택문자가 적음  
  
        ch=str[i]  
        pp=stk.pop()  
        if(ch != pp) : return False # 서로 안맞음.  
    if(not stk.isEmpty()) : return False #스택이 남았음.  
    return True
```

1. \$ 전까지 w를 스택에 넣는다.

Problem 04

LinkedList 타입의 객체 a의 내용을 그대로 또 다른 LinkedList 타입의 객체 b로 복사하는 코드를 작성하시오. (a의 레퍼런스 값을 b로 복사하여 두 변수가 같은 레퍼런스를 가져 내용이 같아지도록 하라는 것이 아니라 내용 복사를 요구하는 것이다.)

1. a의 내용을 b로 복사한다.

```
def copyStack(a,b):  
    temp=LinkedList()  
    while (not a.isEmpty()) :  
        temp.push(a.pop())  
    while (not temp.isEmpty()) :  
        b.push(temp.pop())
```


(1) **다비칩**

필요법 $\rightarrow$ ① 개수 세는 방식

```
stk = LinkedStack()
for index in range(len(s)):
    ch = s[index]
    if(ch == '(' ) :
        stk.push(ch)
    elif(ch == ')') :
        if(stk.IsEmpty()) : return False;
        else :stk.pop()
if(stk.IsEmpty()) :
    return True
else :
    return False
```

2011.10.10

출제 예상 문제

```
symbol = ['(', '{', '[' ]
symbol2 = [')', '}', ']' ]
stk = LinkedStack()
for index in range(len(s)):
    ch = s[index]
```

```

if(ch in symbol ) :
    stk.push(ch)
elif(ch in symbol2) :
    if(stk.IsEmpty()) :
        return False;
    #추가 로직
    top = stack.pop()
    if(ch != top) :
        return False

if(stk.IsEmpty()) :
    return True
else : # 남아있는 경우
    return False

```

3. 스택 예제 2

Problem 01

LinkedListBasic을 이용하여 스택을 구현한다고 했을 때, 맨 앞을 top으로 사용하는 경우와 맨 뒤를 top으로 사용하는 경우에 대해 push, pop의 시간복잡도를 구하시오.



앞 Push : $O(1)$
 앞 Pop : $O(1)$
 뒤 Push : $O(n)$ # 끝까지 탐색해야하니까.
 뒤 Pop : $O(n)$ # 끝까지 탐색해야하니까.

Problem 02

스택의 모든 원소를 제거하는 재귀 함수 clearStack()을 작성하시오.

```

def clearStack(s) : # 재귀 방식
    if (s.isEmpty()) :
        return 0
    s.pop()
    clearStack(s)

```

Problem 03

스택을 이용하여 리스트의 원소들을 역순으로 바꾸는 함수 reverseList(L)를 작성

하시오



#여기서는 배열리스트이다.

```
def reverseList(L):
    s=ListStack()
    while(L.isEmpty()) :
        s.push(L.pop())
    for i in range(len(L)):
        L.append(s.pop())
```

Problem 04

문자열을 입력으로 받아 인접한 같은 문자를 제거한 문자열을 반환하는 함수

removeAdjacent(s)를 작성하시오. ⇒ stack 이용. 저장시 Top과 같으면 안넣음.

```
def removeAdjacent(s) :
    stk = ListStack()
    for i in range(len(s)):
        ch = s[i]
        if (stk.isEmpty()) :
            stk.push(ch)
        elif (stk.Top() == ch ) :
            stk.pop()
    result = ""
    while not stk.isEmpty() :
        result= stk.pop() + result
    return result
```

3. (24/1)

Problem 05

stack 이용.

문자열이 좌우 대칭 (palindrome) 인지 검사하는 함수 isPalindrome(s) 함수를 작성

하시오.

⇒ abcba는 palindrome 이고 abcb는 아님

스택에 넣고 top과 문자열 비교

```
def isPalindrome(s) :
    stk = ListStack()
    for i in range(len(s)) :
        ch=s[i]
```

```

stk.push(ch)
for i in range(len(s)) :
    ch=s[i]
    if(ch != stk.top()):
        return false
    stk.pop()
return True

```

Problem 06

스택에서 특정 값 x 를 모두 제거하는 함수 `removeAll(s, x)`를 작성하시오.

```

def removeAll(s,x):
    temp = ListStack()
    while not s.isEmpty() :
        ch=s.pop()
        if(ch != x ) :
            temp.push(ch)
    while not temp.isEmpty() :
        s.push(temp.pop())

```

Problem 07

$\{1,2,3, \dots, n\}$ 의 모든 순열 (permutation)을 출력하는 함수를 작성하시오.

```

def permutations_stack(n):
    # (고른것, 남은것) 이런 형태로 스택에 담는다.
    s=ListStack()
    s.push((0,list(range(1,n+1))))

    while not s.isEmpty() :
        perm, remaining = s.pop()
        if len(remaining)==0 :
            print(perm)
        else :
            for i in range(len(remaining)-1,-1,-1)
                x=remaining[i]
                new_perm=perm + [x]
                new_remaining = remaining[:i]+remaining[i+1:]
                #remaining에서 하나만 빼서 perm에 넣음
                stack.append((new_perm, new_remaining))

```

0/123 넣고 시작

top pop (0/123)

1/23, 2/13, 3/12 만들어서 push

top pop (3/12)

```
top pop (1/23)
12/3, 13/2 만들고 push
top pop (13/2)
132 만들고 print
top pop (12/3)
123 만들고 print
stack= [] 남음
```

Problem 01

```
class ListQueue:
    def __init__(self):
        self.__queue = []

    def enqueue(self, x):
        self.__queue.append(x)

    def dequeue(self):
        return self.__queue.pop()

    def front(self):
        if self.isEmpty():
            return None
        else:
            return self.__queue[0]

    def isEmpty(self) -> bool:
        return len(self.__queue) == 0

    def dequeueAll(self):
        self.__queue.clear()

    def printQueue(self):
        print("Queue from front:", end=' ')
        for i in range(len(self.__queue)):
            print(self.__queue[i], end=' ')
        print()
```

성출제예안.

Problem 02

입력으로 들어온 문자열이 다음 집합의 원소인지 체크하는 코드를 큐를 이용해서 작성하시오.



abc \$ abc 이런건가?

{w\$w| w: 문자열}

```
#앞 W만 큐에 넣고
#뒤 w는 문자열에서 접근.
def checkStr(s) :
    q=ListQueue()
    dollarIndex=-1
    for i in range(len(s)):
        if s[i] == '$' :
            dollarIndex = i
            break
        q.enqueue
    for i in range(dollarIndex+1, len(s),1) :
        if q.isEmpty() :
            return False
        if(s[i] != q.dequeue()) :
            return False
    if q.isEmpty() :
        return True
    else :
        return False
```

큐 a → 큐 c에 옮긴다.

큐 c에서 → 큐 a와 큐 b에 넣는다.

Problem 03

LinkedList 타입의 객체 a의 내용을 그대로 또 다른 LinkedList 타입의 객체 b로 복사하는 코드를 작성하시오. (a의 레퍼런스 값을 b로 복사하여 두 변수가 같은 레퍼런스를 가져 내용이 같아지도록 하라는 것이 아니라 내용 복사를 요구하는 것이다.)

def copyQueue(a,b):

```
# a를 임시 큐c에 옮기고, c에서 a,b로 복사.
def copyQueue(a,b):
    c=LinkedList()
    while(not a.isEmpty()):
        c.enqueue(a.dequeue())
    b.dequeueAll()
    while (not c.isEmpty()):
        temp = c.dequeue()
        a.enqueue(temp)
```

b.enqueue(temp)

Problem 04

비효율적이지만 2개의 큐를 사용하여 스택의 push()와 pop() 알고리즘을 작성하

시오.

큐는 스택역할 ← 1 | 2 | 3 → 큐가. 1 | 2 | 3
큐는 temp역할 ← 4 | 1 | 1 | 2 | 3
큐에 스택순서로 정리해서 넣겠다는 것.

#큐1은 스택역할
#큐2는 temp 역할
#push이면 temp에 넣고 q1의 값을 다 넣고, temp를 다시 q1에 다시 넣어
#pop이면 그냥 큐 dequeue
#큐에 스택순서로 정리해서 넣겠다는 것이다.

```
class QueueStack :  
    def __init__(self) :  
        q1=ListQueue()  
        q2=ListQueue()  
    def push(self,x):  
        self.q2.enqueue(x)  
        while(not q1.isEmpty()) :  
            q2.enqueue(q1.dequeue())  
        self.q1,self.q2 = self.q2, self.q1  
    def pop(self) :  
        return self.q1.dequeue()
```

Problem 05

비효율적이지만 2개의 스택을 사용하여 큐의 enqueue()와 dequeue() 알고리즘을 작성하시오.

넣는 곳 → 1 | 3 | 2 |
빼는 곳 → 1 | 2 |

#s1은 넣는 스택
#s2는 빼는 곳

```
class stackQueue :  
    def __init__ :  
        s1=ListStack  
        s2=ListStack  
    def enqueue (self,x) :  
        self.s1.append(x)  
    def dequeue (self) :  
        if(s2.isEmpty()) :  
            while(not s1.isEmpty()) :
```

```
s2.push(s1.pop())  
return s2.pop()
```

Problem 06



이 장에서 배운 큐에서 enqueue()는 항상 큐의 맨 뒤에, dequeue()는 항상 큐의 맨 앞에서 이루어진다. Deque는 이의 변형으로 enqueue()와 dequeue()가 큐의 맨 앞과 맨 뒤에서 다 가능하다. [코드 7-13]의 클래스 환경에서 이를 가장 효율적으로 구현한다고 하자. 큐의 사이즈가 n 일 때, enqueue()와 dequeue()의 수행 시간은 각각 어떻게 되겠는가?

```
#앞 enqueue  
- O(1) : tail.next  
#앞 dequeue  
- O(1) : tail.next  
  
#뒤 enqueue  
- O(1) : tail 다음에 삽입  
#뒤 dequeue  
- O(n) : tail 앞 노드를 찾으려면 서칭 필요.
```

Problem 07

06번 문제에서 deque를 원형 연결 리스트가 아닌 [코드 5-15]의 LinkedListBasic 객체를 사용한다면 enqueue()와 dequeue()의 수행 시간은 각각 어떻게 되겠는가

```
#head만 있는 경우  
#앞 enqueue  
- O(1) : head 사용  
#앞 dequeue  
- O(1) : head 사용  
  
#뒤 enqueue  
- O(n) : 탐색 필요  
#뒤 dequeue  
- O(n) : 탐색 필요
```


Problem 08

다음은 클래스 ListQueue의 파이썬 코드다. 이를 06번 문제의 자료구조 Deque를 구현하는 클래스 Deque로 바꾸어 보시오. 필요한 메서드를 추가하시오.

대형
앞뒤여부 추가필요

```
class ListQueue:
    def __init__(self):
        self.__queue = []

    def enqueue(self, x):
        self.__queue.append(x)

    def dequeue(self):
        return self.__queue.pop(0)

    def front(self):
        if self.isEmpty():
            return None
        else:
            return self.__queue[0]

    def isEmpty(self) -> bool:
        return len(self.__queue) == 0

    def dequeueAll(self):
        self.__queue.clear()

    def printQueue(self):
        print("Queue from front:", end=' ')
        for i in range(len(self.__queue)):
            print(self.__queue[i], end=' ')
        print()
```

```
def enqueueTail(self, x):
    self.__queue.append(x)

def dequeueTail(self, x):
    self.__queue.pop(-1)

def enqueueFront(self, x):
    self.__queue.insert(0, x)

def dequeueFront(self, x):
    self.__queue.pop(0)
```